

نسخه چاپی 

پولکی

پولکی چیست؟ ⇨

«پولکی» کتابخانه‌ای نو برای پیاده‌سازی پرداخت درون برنامه بازار است که با زبان برنامه نویسی Kotlin توسعه داده شده و از چارچوب ReactiveX پشتیبانی می‌کند. هدف از پیاده‌سازی این کتابخانه، بهبود فرآیند پیاده‌سازی پرداخت درون برنامه ای بازار برای توسعه دهندگان بازار است.

استفاده از پولکی ⇨

۱: افزودن کتابخانه به پروژه ⇨

برای شروع، کتابخانه پولکی را به پروژه خود در فایل build.gradle اضافه کنید:

```
dependencies {
    implementation "com.github.cafebazaar.Poolakey:poolakey:[latest_version]"
}
```

کپی کردن

سپس باید jitpack را به فایل build.gradle اضافه کنید:

```
repositories {
    google()
    jcenter()
    maven { url 'https://jitpack.io' }
}
```

کپی کردن

مطمئن شوید که از آخرین نسخه پولکی استفاده می‌کنید:

<https://github.com/PHELAT/Poolakey/releases>

۲: ساخت و ساز ⇨

ساختار پرداخت درون برنامه ای به شکلی‌ای است که برنامه شما باید مستقیماً با برنامه بازار صحبت کند تا بتواند عملیات خرید و مواردی از این دست را انجام دهد. برای صحبت کردن با بازار ابتدا باید به بازار متصل شوید. برای انجام این کار، وارد فایل Activity یا Fragment خود شوید و در متد onCreate یک نمونه از کلاس SecurityCheck درست کنید:

```
val localSecurityCheck = SecurityCheck.Enable(
    rsaPublicKey = "KEY"
)
```

کپی کردن

به جای مقدار KEY باید از کلید برنامه خود استفاده کنید، برای دریافت کلید برنامه به پنل پیشخوان مراجعه کنید و در صفحه اطلاعات برنامه، وارد تب «پرداخت درون برنامه ای» شوید و کلید را از آنجا کپی کنید و به جای مقدار KEY در نمونه کد بالا قرار دهید.

در مرحله بعد باید یک نمونه از کلاس PaymentConfiguration درست کنید و localSecurityCheck را به آن تزریق کنید:

```
val paymentConfiguration = PaymentConfiguration(
    localSecurityCheck = localSecurityCheck
)
```

کپی کردن

و در آخرین مرحله باید یک نمونه از کلاس Payment درست کنید:

```
val payment = Payment(context = this, config = paymentConfiguration)
```

کپی کردن

کلاس Payment پل ارتباطی بین برنامه شما و بازار است و از این کلاس می‌توانید برای اتصال به بازار، خرید یک محصول و موارد مشابه استفاده کنید.

۳. اتصال به بازار ⇨

برای اتصال به بازار، باید از فانکشن connect موجود در کلاس Payment استفاده کنید:

```
val paymentConnection = payment.connect {
    connectionSucceed {
        ...
    }
    connectionFailed { throwable ->
        ...
    }
    disconnected {
        ...
    }
}
```

کپی کردن

در بدنه فانکشن connect به سه callback دسترسی دارید:

- connectionSucceed وقتی صدا زده می‌شود که اتصال به بازار برقرار شود
- connectionFailed وقتی صدا زده می‌شود که اتصال به بازار برقرار نشود
- disconnected وقتی صدا زده می‌شود که طبق رویدادی ارتباط برنامه شما با بازار قطع شود

همان‌طور که می‌بینید، در connectionFailed به یک نمونه از throwable دسترسی دارید که به کمک آن می‌توان متوجه مشکلی که باعث عدم اتصال برنامه شما به بازار شده است شد.

۴. قطع اتصال با بازار ⇨

برای جلوگیری از مشکلاتی همچون Memory Leak باید در متد onDestroy از Activity یا Fragment خود، اقدام به قطع اتصال با بازار کنید:

```
override fun onDestroy() {
    paymentConnection.disconnect()
    super.onDestroy()
}
```

کپی کردن

۵. شروع روند خرید یک محصول ⇨

برای شروع روند خرید یک محصول، باید ابتدا یک نمونه از کلاس PurchaseRequest ساخته شود:

```
val purchaseRequest = PurchaseRequest(
    productId = "PRODUCT_ID",
    payload = "PAYLOAD"
)
```

کپی کردن

به جای مقدار PRODUCT_ID باید مقدار شناسه محصولی که قصد خرید آن را دارید را جایگزین کنید. برای دریافت مقدار شناسه محصول مورد نظر، به پیشخان توسعه‌دهندگان مراجعه کنید و در صفحه اطلاعات برنامه، در تب «پرداخت درون برنامه ای» یک محصول جدید بسازید و شناسه‌ای برای آن در نظر بگیرید. یا این که می‌توانید، شناسه محصول‌هایی که پیش از این ساخته‌اید را جایگزین مقدار PRODUCT_ID کنید.

سپس با استفاده از فانکشن purchaseProduct از کلاس Payment روند خرید محصول مورد نظر را شروع کنید:

```
payment.purchaseProduct(
    registry = activityResultRegistry,
    request = purchaseRequest
) {
    purchaseFlowBegan {
        ...
    }
    failedToBeginFlow { throwable ->
        ...
    }
}
```

```

    }
    purchaseSucceed { purchaseEntity ->
        ...
    }
    purchaseCanceled {
        ...
    }
    purchaseFailed { throwable ->
        ...
    }
}

```

کپی کردن

در بدنه فانکشن purchaseProduct به callback های زیر دسترسی دارید:

- purchaseFlowBegan وقتی صدا زده می‌شود که فرایند هدایت کاربر به صفحه خرید بازار به درستی انجام شود.
- failedToBeginFlow وقتی صدا زده می‌شود که فرایند هدایت کاربر به صفحه خرید بازار به درستی انجام نشود. همانطور که می‌بینید، در failedToBeginFlow به یک نمونه از throwable دسترسی دارید که به کمک آن می‌توان متوجه مشکلی که در شروع فرایند خرید شده است شد.
- purchaseSucceed وقتی صدا زده می‌شود که خرید با موفقیت انجام شود، که در این صورت از طریق purchaseEntity می‌توانید به اطلاعات خرید دست پیدا کنید. توجه کنید که در صورت موفقیت‌آمیز بودن خرید، بسیار پیشنهاد می‌شود که از طریق API پرداخت بازار، خودتان صحت پرداخت را نیز بررسی کنید.
- purchaseCanceled وقتی صدا زده می‌شود که کاربر از خرید منصرف شود.
- purchaseFailed وقتی صدا زده می‌شود که مشکلی در خرید وجود داشته باشد. این مشکل می‌تواند از جنس صحت نداشتن اطلاعات خرید و مشکلات مشابه باشد. در این صورت به کمک نمونه throwable موجود در این callback می‌توانید از اشکال به‌وجود آمده مطلع شوید.

۶: شروع روند اشتراک یک محصول ⇨

برای اشتراک یک محصول، دقیقاً مثل شروع خرید یک محصول عمل کنید و به جای استفاده از فانکشن purchaseProduct از فانکشن subscribeProduct موجود در کلاس Payment استفاده کنید.

۷: مصرف کردن خرید کاربر ⇨

وقتی کاربر یکی از محصولات دیجیتالی برنامه شما را خریداری می‌کند، مالک یک نسخه محصول مورد نظر می‌شود و در صورتی که شما کاربر را مجدداً به صفحه پرداخت هدایت کنید، کاربر نمی‌تواند دوباره خرید خود را انجام دهد و با اطلاعات خرید قبلی به برنامه شما باز می‌گردد (purchaseSucceed). اما شما می‌توانید برای محصول مورد نظر خود تاریخ مصرف و یا شرایط خاصی را در نظر بگیرید تا خرید مصرف شود. در این صورت کاربر می‌تواند مجدداً خرید خود را انجام دهد. برای مصرف کردن یک خرید از فانکشن consumeProduct موجود در کلاس Payment استفاده کنید:

```

payment.consumeProduct("PURCHASE_TOKEN") {
    consumeSucceed {
        ...
    }
    consumeFailed { throwable ->
        ...
    }
}

```

کپی کردن

همان‌طور که می‌بینید، در بدنه consumeProduct به دو callback دسترسی دارید:

- consumeSucceed وقتی صدا زده می‌شود که عملیات مصرف کردن خرید با موفقیت انجام شود.
 - consumeFailed وقتی صدا زده می‌شود که عملیات مصرف کردن خرید انجام نشود، در این شرایط شما می‌توانید با استفاده از نمونه throwable موجود در این callback متوجه مشکل موجود شوید.
- توجه کنید به‌جای مقدار PURCHASE_TOKEN در نمونه کد بالا، باید مقدار واقعی توکن خریدی که می‌خواهید مصرف شود را قرار دهید. دسترسی به توکن خرید، از طریق کلاس PurchaseEntity امکان‌پذیر است. وقتی کاربر خرید انجام می‌دهد، پولکی به شما یک PurchaseEntity در purchaseSucceed می‌دهد. همچنین می‌توانید تمام خریدهای کاربر که در برنامه شما انجام داده است را از طریق فانکشن getPurchasedProducts بدست بیاورید که در این صورت، پولکی به شما لیستی از PurchaseEntity می‌دهد که از این طریق نیز می‌توان به توکن خرید دسترسی پیدا کرد.

توجه: محصولات اشتراکی را نمی‌توانید مصرف کنید.

توجه: به این نکته توجه داشته باشید که قبل از کار با consumeProduct باید مطمئن شوید که کاربر وارد حساب بازار خود شده است، در غیر این‌صورت consumeFailed صدا زده می‌شود.

۸: دریافت خریدهای مربوط به پرداخت درون برنامه ای کاربر در برنامه شما ⇒

شما می‌توانید با استفاده از فانکشن `getPurchasedProducts` موجود در کلاس `Payment` از خریدهای کاربر که در برنامه شما انجام داده است، مطلع شوید:

```
payment.getPurchasedProducts {
    querySucceed { purchasedProducts ->
        ...
    }
    queryFailed { throwable ->
        ...
    }
}
```

کپی کردن

همانطور که می‌بینید، در بدنه `getPurchasedProducts` به دو `callback` دسترسی دارید:

- `querySucceed` وقتی صدا زده می‌شود که ارتباط با بازار به خوبی برقرار شود و مشکلی در دریافت لیست خریدها وجود نداشته باشد. در این صورت، با استفاده از `purchasedProducts` که لیستی از `PurchaseInfo` است می‌توانید به خریدهای کاربر دسترسی پیدا کنید.
- `queryFailed` وقتی صدا زده می‌شود که ارتباط با بازار برقرار نشود یا مشکلی در دریافت لیست خریدها وجود داشته باشد. در این صورت، می‌توانید با استفاده از نمونه `throwable` موجود در این `callback` متوجه مشکل به‌وجود آمده شوید.

توجه: به این نکته توجه داشته باشید قبل از کار با `getPurchasedProducts` باید مطمئن شوید که کاربر وارد حساب بازار خود شده است، در غیر این‌صورت `queryFailed` صدا زده می‌شود.

۹: دریافت اشتراک‌های کاربر در برنامه شما ⇒

شما می‌توانید با استفاده از فانکشن `getSubscribedProducts` موجود در کلاس `Payment` از محصولات که کاربر اشتراک آن‌ها را در برنامه شما خریداری کرده، مطلع شوید:

```
payment.getSubscribedProducts {
    querySucceed { purchasedProducts ->
        ...
    }
    queryFailed { throwable ->
        ...
    }
}
```

کپی کردن

همانطور که می‌بینید، در بدنه `getSubscribedProducts` به دو `callback` دسترسی دارید:

- `querySucceed` وقتی صدا زده می‌شود که ارتباط با بازار به خوبی برقرار شود و مشکلی در دریافت لیست اشتراک‌ها وجود نداشته باشد که در این صورت، با استفاده از `purchasedProducts` که لیستی از `PurchaseInfo` است می‌توانید به اشتراک‌های کاربر دسترسی پیدا کنید.
- `queryFailed` وقتی صدا زده می‌شود که ارتباط با بازار برقرار نشود یا مشکلی در دریافت لیست اشتراک‌ها وجود داشته باشد، که در این شرایط، می‌توانید با استفاده از نمونه `throwable` موجود در این `callback` متوجه مشکل به‌وجود آمده شوید.

توجه: قبل از کار با `getSubscribedProducts` باید مطمئن شوید که کاربر وارد حساب بازار خود شده است، در غیر این‌صورت `queryFailed` صدا زده می‌شود.

۱۰: بررسی وضعیت اشتراک آزمایشی برای کاربر ⇒

شما می‌توانید با استفاده از فانکشن `checkTrialSubscription` موجود در کلاس `Payment` از وضعیت امکان تعلق اشتراک آزمایشی به کاربر در برنامه خود مطلع شوید:

```
payment.checkTrialSubscription {
    checkTrialSubscriptionSucceed { trialSubscriptionInfo ->

    }

    checkTrialSubscriptionFailed { throwable ->

    }
}
```

کپی کردن

همان‌طور که می‌بینید، در بدنه checkTrialSubscription به دو callback دسترسی خواهید داشت:

- checkTrialSubscriptionSucceed وقتی صدا زده می‌شود که ارتباط با بازار به خوبی برقرار شود و مشکلی در دریافت وضعیت اشتراک آزمایشی وجود نداشته باشد که در این صورت، با استفاده از trialSubscriptionInfo می‌توانید به وضعیت امکان تعلق اشتراک آزمایشی کاربر دسترسی پیدا کنید. trialSubscriptionInfo شامل فیلدهای زیر است:
- isAvailable: (که در صورتی که فعال باشد true و در غیر این صورت false خواهد بود)، این فیلد در صورت فعال بودن نشانگر این است که در صورت خرید اشتراک توسط کاربر، اشتراک آزمایشی بجای اشتراک خریداری شده برای او فعال خواهد شد و پس از اتمام اشتراک آزمایشی با اشتراک اول تمدید خواهد شد.
- trialPeriodDays: طول اشتراک آزمایشی که به کاربر تعلق خواهد گرفت.
- checkTrialSubscriptionFailed وقتی صدا زده می‌شود که ارتباط با بازار برقرار نشود یا مشکلی در دریافت وضعیت اشتراک آزمایشی وجود داشته باشد، که در این شرایط، می‌توانید با استفاده از نمونه throwable موجود در این Callback، متوجه مشکل به‌وجود آمده شوید.

نکات مهم:

قبل از کار با checkTrialSubscription باید مطمئن شوید که کاربر وارد حساب بازار خود شده است، در غیر این‌صورت checkTrialSubscriptionFailed صدا زده می‌شود.

برای بررسی وضعیت اشتراک کاربر پس از فعال‌سازی اشتراک آزمایشی می‌توانید از قسمت «دریافت اشتراک‌های کاربر در برنامه شما» استفاده کنید.

پرداخت با یونیتی

با استفاده از دستور زیر می‌توانید از پلاگین پرداخت درون برنامه یونیتی برای پولکی استفاده کنید.

```
var connectionResult = await payment.Connect();
var purchaseResult = await payment.Purchase("productID");
var consumeResult = await payment.Consume(purchaseResult.data.purchaseToken);
```

کپی کردن

توجه: کتابخانه پولکی برای پروژه‌های یونیتی 2020 و بالاتر قابل اجرا می‌باشد.

سوالات متداول

۱. پولکی چیست و چه کاربردی دارد؟

پولکی یک کتابخانه برای پیاده‌سازی پرداخت درون‌برنامه‌ای بازار است که با زبان برنامه‌نویسی Kotlin توسعه یافته است. این کتابخانه هدف دارد فرآیند پیاده‌سازی پرداخت‌ها در برنامه‌ها را برای توسعه‌دهندگان ساده و سریع‌تر کند. همچنین، از چارچوب ReactiveX پشتیبانی می‌کند که مدیریت عملیات‌ها را به صورت غیرهمزمان آسان‌تر می‌نماید.

۲. چگونه می‌توان کتابخانه پولکی را به یک پروژه افزود؟

برای افزودن کتابخانه پولکی به پروژه:

1. در فایل build.gradle این وابستگی را اضافه کنید:

```
implementation "com.github.cafebazaar.Poolakey:poolakey:[latest_version]"
```

کپی کردن

2. مخزن jitpack را به فایل build.gradle اضافه کنید:

```
repositories {
    google()
    jcenter()
    maven { url 'https://jitpack.io' }
}
```

کپی کردن

3. مطمئن شوید که از آخرین نسخه پولکی استفاده می‌کنید. نسخه‌های جدید را می‌توانید از این لینک پیدا کنید:

[لینک نسخه‌های پولکی](#)

۳. کلید RSA چیست و چگونه باید آن را تنظیم کرد؟

کلید RSA برای امنیت انتقال اطلاعات و تأیید صحت خرید استفاده می‌شود. برای تنظیم کلید RSA:

1. وارد پیشخوان توسعه‌دهندگان شوید و به تب پرداخت درون‌برنامه‌ای بروید.
2. کلید RSA برنامه را کپی کنید و آن را به جای مقدار "KEY" در کدی مانند زیر جای‌گذاری کنید:

```
val localSecurityCheck = SecurityCheck.Enable(
    rsaPublicKey = "YOUR_RSA_PUBLIC_KEY"
)
```

کپی کردن

۴. چگونه می‌توان بین برنامه و کافه بازار ارتباط برقرار کرد؟

برای اتصال به کافه بازار:

1. ابتدا یک نمونه از کلاس Payment ایجاد کنید:

```
val payment = Payment(context = this, config = paymentConfiguration)
```

کپی کردن

2. از متد connect برای اتصال به بازار استفاده کنید:

```
val paymentConnection = payment.connect {
    connectionSucceed { /* اتصال موفق */ }
    connectionFailed { throwable -> /* اتصال ناموفق */ }
    disconnected { /* قطع اتصال */ }
}
```

کپی کردن

- connectionSucceed : وقتی ارتباط برقرار شود، این callback صدا زده می‌شود.
- connectionFailed : وقتی اتصال با بازار با شکست مواجه شود، دلیل خطا (Throwable) در این callback در دسترس است.
- disconnected : وقتی ارتباط با بازار قطع شود، این callback صدا زده می‌شود.

۵. چگونه می‌توان یک محصول را برای خرید فعال کرد؟

برای شروع خرید یک محصول:

1. شناسه محصول (PRODUCT_ID) و اطلاعات اضافی (PAYLOAD) را مشخص کنید:

```
val purchaseRequest = PurchaseRequest(
    productId = "PRODUCT_ID",
    payload = "PAYLOAD"
)
```

کپی کردن

2. فرآیند خرید را آغاز کنید:

```
payment.purchaseProduct(
    registry = activityResultRegistry,
    request = purchaseRequest
) {
    purchaseFlowBegan { /* خرید آغاز شد */ }
    failedToBeginFlow { throwable -> /* آغاز نشد */ }
    purchaseSucceed { purchaseEntity -> /* خرید موفق */ }
    purchaseCanceled { /* کاربر انصراف داد */ }
    purchaseFailed { throwable -> /* خطا در خرید */ }
}
```

کپی کردن

۶. چگونه می‌توان اشتراک یک محصول را تنظیم کرد؟

برای خرید اشتراک محصول، از متد subscribeProduct استفاده کنید. این متد مشابه متد purchaseProduct کار می‌کند، اما به جای خرید مستقیم، فرایند اشتراک محصول را آغاز می‌کند.

۷. مفهوم "مصرف کردن خرید" چیست و چه زمانی باید از آن استفاده کرد؟

اگر یک محصول دیجیتالی خریداری شده (مثل الماس یا اعتبار درون برنامه‌ای) نیاز به دوباره خریدن دارد، باید خرید قبلی مصرف شود تا کاربر بتواند دوباره آن محصول را خریداری کند.

برای مصرف خرید، از متد consumeProduct استفاده کنید:

```
payment.consumeProduct("PURCHASE_TOKEN") {
    consumeSucceed { /* موفقیت‌آمیز */ }
    consumeFailed { throwable -> /* ناموفق */ }
}
```

کپی کردن

🔔 نکته: محصولات اشتراکی را نمی‌توان مصرف کرد.

۸. چگونه می‌توان لیست خریده‌ها یا اشتراک‌های کاربر را دریافت کرد؟

- برای دریافت خریده‌های عادی:

از متد `getPurchasedProducts` استفاده کنید:

```
payment.getPurchasedProducts {
    querySucceed { purchasedProducts -> /* لیست خریده‌ها */ }
    queryFailed { throwable -> /* خطا */ }
}
```

کپی کردن

- برای دریافت اشتراک‌ها:

از متد `getSubscribedProducts` استفاده کنید:

```
payment.getSubscribedProducts {
    querySucceed { purchasedProducts -> /* لیست اشتراک‌ها */ }
    queryFailed { throwable -> /* خطا */ }
}
```

کپی کردن

🔔 نکته: مطمئن شوید کاربر به حساب بازار خود وارد شده باشد؛ در غیر این صورت، درخواست با خطا مواجه خواهد شد.

۹. وضعیت اشتراک آزمایشی چگونه بررسی می‌شود؟

برای بررسی امکان تعلق گرفتن اشتراک آزمایشی به کاربر:

```
payment.checkTrialSubscription {
    checkTrialSubscriptionSucceed { trialSubscriptionInfo ->
        /* اطلاعات اشتراک آزمایشی */
    }
    checkTrialSubscriptionFailed { throwable ->
        /* خطا در بررسی */
    }
}
```

کپی کردن

- `isAvailable` : بررسی می‌کند که آیا اشتراک آزمایشی برای کاربر فعال است.
- `trialPeriodDays` : تعداد روزهایی که اشتراک آزمایشی در دسترس خواهد بود.

۱۰. چگونه از خطاهای احتمالی در فرایند پرداخت مطلع شویم؟

پولکی در تمامی متدهای خود از `callback`هایی مانند `failedToBeginFlow` , `queryFailed` , `purchaseFailed` و ... استفاده می‌کند. این `callback`ها نمونه‌ای از `Throwable` را ارائه می‌دهند که جزئیات خطا را نمایش می‌دهد. می‌توانید از این اطلاعات برای گزارش خطاها یا رفع اشکال استفاده کنید.

۱۱. آیا پولکی برای پروژه‌های مبتنی بر یونیتی نیز قابل استفاده است؟

بله. پولکی برای پروژه‌های یونیتی نسخه 2020 و بالاتر به‌خوبی پشتیبانی می‌شود. برای شروع پرداخت در یونیتی، می‌توانید از متدهای زیر استفاده کنید:

```
var connectionResult = await payment.Connect();
var purchaseResult = await payment.Purchase("productID");
var consumeResult = await payment.Consume(purchaseResult.data.purchaseToken);
```

کپی کردن

۱۲. چگونه از Memory Leak هنگام استفاده از پولکی جلوگیری کنیم؟

برای جلوگیری از Memory Leak، ارتباط با بازار در متد onDestroy قطع کنید:

```
override fun onDestroy() {
    paymentConnection?.disconnect()
    super.onDestroy()
}
```

کپی کردن

۱۳. چگونه از امنیت داده‌ها در پولکی اطمینان حاصل کنیم؟

- با استفاده از کلید RSA عمومی، صحت پرداخت‌ها را بررسی کنید.
- پس از هر خرید موفق، صحت آن را از طریق API بازار در سرور خود تأیید کنید.
- از ذخیره‌سازی اطلاعات حساس مانند PURCHASE_TOKEN در بخش محلی برنامه خودداری کنید.

۱۴. API پرداخت بازار چیست و چه کاربردی دارد؟

API پرداخت بازار به شما این امکان را می‌دهد که صحت پرداخت‌های انجام‌شده در برنامه خود را از سمت سرور تأیید کنید. این فرایند برای افزایش امنیت و جلوگیری از تخلف‌های پرداخت بسیار مفید است.

بیشتر بخوانید:

- خطای نصب در پیشخوان بازار چگونه کار می‌کند؟
- راهنمای پیشخوان
- معرفی پرداخت

به مطالب ارائه شده چه امتیازی می‌دهید؟

محل نوشتن دیدگاه ...

اگر در مورد محتوا نظر یا پیشنهادی دارید لطفاً برای ما بنویسید.

محل نوشتن دیدگاه ...

ارسال دیدگاه

آخرین به‌روزرسانی: ۱۴۰۳-۱۰-۲۵ ۰۹:۴۵

زمان انتشار: ۱۴۰۰-۰۳-۲۸ ۱۰:۰۸

فهرست مطالب

پولکی چیست؟

استفاده از پولکی

۱: افزودن کتابخانه به پروژه

۲: ساخت و ساز

۳: اتصال به بازار

۴: قطع اتصال با بازار

۵: شروع روند خرید یک محصول

۶: شروع روند اشتراک یک محصول

۷: مصرف کردن خرید کاربر

۸: دریافت خریدهای مربوط به پرداخت درون برنامه ای کاربر در برنامه شما

۹: دریافت اشتراکهای کاربر در برنامه شما

۱۰: بررسی وضعیت اشتراک آزمایشی برای کاربر

پرداخت با یونیتی

سوالات متداول